

## Arquitectura de Software — Unidad 1 Actividad práctica — Tema 3: Fundamentos y objetivos de la Arquitectura de Software

### 1. Comprensión de los objetivos arquitectónicos

- ¿Qué se espera lograr al diseñar una buena arquitectura de software?  
Al diseñar una buena arquitectura se busca no solo organizar el código, también responder a los objetivos estratégicos que aseguran la calidad, mantenibilidad, escalabilidad y viabilidad a largo plazo. Es decir, que el software sea sólido, fácil de evolucionar y responda bien a las necesidades del negocio y de los usuarios.
- ¿Cómo se relacionan los siguientes aspectos con la arquitectura?  
**Rendimiento:** Define cómo fluyen los datos, cómo se comunican los componentes y cómo se gestionan los recursos. Una buena arquitectura permite que el sistema sea rápido y eficiente, pero decisiones mal tomadas pueden volverlo pesado.  
**Escalabilidad:** Está directamente ligada a la arquitectura, ya que define si el sistema podrá crecer cuando aumenten los usuarios o las operaciones.  
**Seguridad:** Establece dónde y cómo se aplican controles como autenticación, autorización, cifrado y protección de datos.
- Describe cómo estos objetivos pueden entrar en conflicto entre sí en ciertos proyectos, y cómo equilibrarlos.
- Estos objetivos a veces chocan entre sí. Por ejemplo, más seguridad puede hacer el sistema más lento, o diseñar para que escale demasiado desde el inicio puede bajar el rendimiento o abrir riesgos de seguridad.  
El equilibrio está en tomar decisiones conscientes según lo que realmente necesita el proyecto: usar cachés para no perder velocidad, aplicar seguridad de forma práctica sin sobrecargar, y planear la escalabilidad de manera gradual. La idea es que la arquitectura encuentre un punto medio que mantenga todo funcionando bien y de forma sostenible.

### 2. Análisis de decisiones arquitectónicas

- ¿Qué significa tomar una “decisión arquitectónica” en el contexto de un sistema de software?  
Tomar una decisión arquitectónica en el contexto de un sistema de software significa elegir, de forma deliberada y con impacto sistémico, entre alternativas técnicas que moldean la estructura, el comportamiento y las cualidades del sistema a lo largo de su ciclo de vida.  
En términos más concretos, es una decisión que:  
Define la estructura global del sistema (por ejemplo, si se usa una arquitectura de microservicios, monolito, capas, event-driven, etc.).  
Restringe o habilita futuras decisiones técnicas y de negocio.  
Impacta en atributos de calidad como escalabilidad, seguridad, rendimiento, mantenibilidad o disponibilidad.

Asigna responsabilidades a componentes o módulos (por ejemplo, qué servicio gestiona la autenticación, qué base de datos almacena datos transaccionales, etc.).

Implica compromisos que son costosos de revertir si se cambian más adelante (por ejemplo, elegir una tecnología específica, un modelo de datos, un protocolo de comunicación, etc.)

- Menciona al menos tres ejemplos reales de decisiones arquitectónicas (tecnologías, patrones, restricciones) que impacten directamente el sistema desde sus primeras fases.

Elección del patrón arquitectónico: Microservicios vs. Monolito Modular  
Decidir si el sistema se estructurará como un monolito modular (un único despliegue con módulos internos bien acotados) o como microservicios (servicios independientes con despliegue y ciclo de vida propios) impacta la escalabilidad, la curva de aprendizaje del equipo y los costos operativos desde el día 0 .

Tecnología de persistencia: Base de datos relacional (PostgreSQL) vs. NoSQL distribuida (Cassandra)

Elegir PostgreSQL favorece la consistencia transaccional y las consultas complejas, mientras que optar por Cassandra permite escalado horizontal masivo y alta disponibilidad, pero limita ciertos tipos de consultas. Esta restricción condiciona el diseño del modelo de dominio y los contratos de API .

Restricción de seguridad: API Gateway centralizado con autenticación basada en JWT + OAuth 2.0

Imponer que todo tráfico externo pase por un API Gateway que valide tokens JWT impone una restricción transversal a todos los servicios. Esto simplifica la gestión de credenciales y reduce la superficie de ataque, pero fuerza a que cada microservicio delegue la autorización al Gateway.

- Selecciona uno de estos ejemplos y analiza:

¿Por qué es una decisión crítica?

Elegir entre microservicios y monolito modular es una decisión arquitectónica crítica porque actúa como la primera y más poderosa restricción que el proyecto se impone a sí mismo: condiciona la forma en que se divide el dominio, la manera en que los equipos se organizan y comunican (por la Ley de Conway), la estrategia de despliegue y la curva de talento que la organización necesitará; además, al ser un cambio extremadamente caro de revertir —porque implica rediseñar despliegues, re factorizar la base de código, reentrenar al personal y reconfigurar toda la infraestructura—, fija de forma casi permanente los límites máximos de escalabilidad horizontal, la latencia que el sistema podrá alcanzar, la superficie de fallo y la complejidad operativa que el negocio deberá asumir día a día; en consecuencia, esta elección define, antes incluso de escribir la

primera línea de código productivo, si el proyecto crecerá con agilidad o se verá frenado por la rigidez de su propia arquitectura.

¿Qué podría pasar si se toma mal?

Si se toma mal la decisión entre microservicios y monolito, el sistema arrastra desde el día uno una trampa de arquitectura: con microservicios innecesarios se multiplican los despliegues, la latencia, la factura cloud y la carga operativa sin beneficio real, hasta que cada release se convierte en una orquestación interminable y los equipos pasan más tiempo depurando redes que añadiendo valor; si, por el contrario, se opta por un monolito cuando el negocio exige crecimiento explosivo, el código se vuelve un “big ball of mud”, las compilaciones duran horas, cualquier cambio rompe diez módulos, las bases de datos se saturan y los picos de tráfico provocan caídas que erosionan la marca y los ingresos, mientras la empresa se ve forzada a una costosa reescritura en plena fase de expansión.

### 3. Evaluación de restricciones reales

- En muchos proyectos se enfrentan restricciones técnicas, físicas y presupuestales. Explica en qué consisten cada una:
  - **Técnicas (ej: compatibilidad, lenguaje, hardware)**  
Son las limitaciones propias de la tecnología que se usa, como la compatibilidad entre sistemas, el lenguaje de programación disponible, la capacidad del hardware o las herramientas que el equipo domina.
  - **Físicas (ej: conectividad, dispositivos, entorno)**  
Tienen que ver con el entorno real donde se usará el sistema. Por ejemplo, la calidad de la conectividad a internet, el tipo de dispositivos donde se ejecutará la aplicación, o incluso condiciones ambientales (espacios, redes locales, energía, etc.).
  - **Presupuestales (ej: licencias, escalabilidad, mantenimiento)**  
Son los límites de dinero y recursos. Incluyen el costo de licencias de software, la inversión necesaria para escalar la solución, los gastos de mantenimiento y soporte, o simplemente el presupuesto disponible para el proyecto.
- Imagina un escenario donde debes diseñar la arquitectura para una app de salud móvil con presupuesto limitado. Menciona al menos tres decisiones arquitectónicas que se verían afectadas por estas restricciones.

Se usarán decisiones claramente arquitectónicas, porque afectarían la base tecnológica, la manera en que los componentes se integran y el rumbo del proyecto a largo plazo.

Decisiones arquitectónicas que se verían afectadas:

**Elección de la base de datos:** en lugar de una solución comercial costosa, se optaría por una base de datos open source como PostgreSQL o MySQL, reduciendo gastos en licencias.

**Infraestructura de despliegue:** en vez de un servidor dedicado o infraestructura propia, se podría usar servicios en la nube con planes gratuitos o de bajo costo (ej: AWS Free Tier, Firebase, Heroku), sacrificando algo de flexibilidad, pero ajustándose al presupuesto.

**Arquitectura de la aplicación:** se podría diseñar como una arquitectura monolítica ligera en lugar de microservicios, ya que estos últimos requieren mayor inversión en infraestructura, monitoreo y personal capacitado.